Taller B1-T1: Arquitectura, Entorno y Extractor

MIAX - BME | Entorno Python, Modelado DataClass y Verificación de Ingesta

1. Entorno Virtual y Dependencias Plug-and-Play (Requisito 4.1)

Creación del entorno local aislado desde la raíz del repositorio (`Taller-T1-B1/`), instalación reproducible de dependencias, registro del paquete en modo editable (*src-layout* configurado en `pyproject.toml`) y ejecución de la suite de tests unitarios:

```
cd /home/sergio/Documents/Master_MIAX_BME/Talleres_Master_MIAX_BME/Taller-T1-B1
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
pip install -e .

# Verificar instalación y lanzar los 19 tests unitarios offline
python -c "import toolkit; print(toolkit.__all__)"
python -m unittest discover -s tests -v
```

2. Estructura Modular en `/src` y Separación por Capas

El paquete `toolkit` sigue una arquitectura estrictamente desacoplada por capas (`data → utils → models → reports`):

```
src/toolkit/
├── __init__.py          # API pública exportada (__all__)
├── logging_utils.py     # Configuración idempotente de logging (setup_logging)
├── reports.py           # Presentación: informes Markdown y gráficos Matplotlib
├── data/                # Ingesta multifuente (base.py, yahoo.py, fred.py)
├── models/              # Dominio (series, portfolio, monte_carlo, metrics)
└── utils/               # Limpieza y validación (cleaning.py, validation.py)
```

3. Ingesta Multifuente Estandarizada y `PriceSeries` (Requisitos 4.2 y 4.3)

Todos los conectores heredan de `BaseDataExtractor` (`YahooFinanceExtractor` para renta variable e índices bursátiles, y `FREDExtractor` para series macroeconómicas de la Reserva Federal con *fallback* automático a `fredgraph.csv`) y normalizan sus salidas mediante `_normalize_close_dataframe()` al esquema canónico `["close", "returns"]` dentro de `PriceSeries` :

```
@dataclass
class PriceSeries:
    ticker: str
    asset_type: str
    data: pd.DataFrame
    mean_return: float = field(init=False)
    std_return: float = field(init=False)

    def __post_init__(self) -> None:
        if "close" not in self.data.columns:
            raise ValueError("Input data must contain a 'close' column.")
```

```
        self.ticker = self.ticker.strip().upper()
        self.data = self.data.copy()  # Copia defensiva sin mutar la entrada
        if "returns" not in self.data.columns:
            self.data["returns"] = self.data["close"].pct_change()
        clean_returns = self.returns
        self.mean_return = float(clean_returns.mean()) if not clean_returns.empty
else 0.0
        self.std_return = float(clean_returns.std()) if not clean_returns.empty
else 0.0
```

4. Validación de Extracción Multifuente y Descarga en Lote

Ejecución de `examples/test_fetch.py` (Yahoo Finance: acciones e índices en lote) y `examples/test_fetch_fred.py` (FRED: series macroeconómicas interoperables). Todos los scripts de `examples/` incluyen resolución automática de `src/` en `sys.path`, por lo que pueden ejecutarse tanto desde la raíz como desde la carpeta `examples/`:

```
(.venv) sergio@sergio-xps-deb:~/Documents/Master_MIAX_BME/Talleres_Master_MIAX_BME/Taller
-T1-B1$ python examples/test_fetch.py
Loaded: AAPL | Mean Return: 0.00066 | Std: 0.02045
Batch item: MSFT (250 trading days)
Batch item: SAN.MC (255 trading days)
Batch item: ^IBEX (255 trading days)
(.venv) sergio@sergio-xps-deb:~/Documents/Master_MIAX_BME/Talleres_Master_MIAX_BME/Taller
-T1-B1$ python examples/test_fetch_fred.py
[MACRO] DFF | Mean: -0.000729 | Std: 0.009114 | Observations: 366
[MACRO] VIXCLS | Mean: +0.006584 | Std: 0.095120 | Observations: 252
[MACRO] DCOILWTICO | Mean: +0.000215 | Std: 0.018940 | Observations: 250
```